

COL100: Introduction to Computer Science

Lab 12: Object-Oriented Programming

April 8, 2025

1 Introduction

1.1 Classes and Objects

In object-oriented programming (OOP), “classes” and “objects” are the building blocks of the code structure. Understanding them is essential for writing efficient and maintainable programs.

1.1.1 Syntax for Defining a Class in Python

Here’s the basic syntax for defining a class in Python:

```
1 class ClassName:
2     def __init__(self, attribute1, attribute2):
3         self.attribute1 = attribute1
4         self.attribute2 = attribute2
5
6     def method1(self):
7         # method logic here
8         pass
```

In the above example, please note the following.

- `__init__` is the constructor method that is automatically called when an object is created from the class. It is used to initialize the object’s attributes.
- `self` refers to the instance of the object being created. It allows the method to access the attributes and methods of the specific object instance.

1.1.2 Example: Creating a Class and Object

Here’s a simple example of defining and creating an object from a class:

```
1 class Car:
2     def __init__(self, make, model, year):
3         self.make = make
4         self.model = model
5         self.year = year
6
7     def get_description(self):
```

```

8         return f"{self.year} {self.make} {self.model}"
9
10 # Create an object of the Car class
11 my_car = Car("Toyota", "Camry", 2020)
12
13 # Accessing object's method
14 print(my_car.get_description()) # Output: "2020 Toyota Camry"

```

In the above example, we do the following.

- We define a `Car` class with an `__init__` method to initialize its attributes (`make`, `model`, `year`).
- We then create an object `my_car` from the `Car` class and print its description using the `get_description()` method.

1.2 Object-Oriented Programming

Here, we will discuss the core concepts of object-oriented programming, i.e., abstraction, encapsulation, inheritance, and polymorphism.

1.2.1 Abstraction

Abstraction is the concept of hiding the complex implementation details and showing only the essential features of an object. In Python, abstraction can be achieved by simply defining a function. It can also be achieved by defining a class with data/attributes and functions/methods without exposing the user to the detailed implementation.

Example:

```

1 class Vehicle:
2     def __init__(self, name, max_speed):
3         self.name = name
4         self.max_speed = max_speed
5
6     def display_info(self):
7         return f"{self.name} can go up to {self.max_speed} km/h."
8
9 # Using the class
10 car = Vehicle("Car", 180)
11 print(car.display_info())

```

Explanation

Here, the user interacts with the `Vehicle` class through the `display_info` method, which abstracts the internal details of how the data is stored (like `name` and `max_speed`). The internal data is not directly exposed, but it is made accessible through methods.

1.2.2 Encapsulation

Encapsulation is the concept of bundling the data (attributes) and methods (functions) into a single unit (class). It is also possible to restrict access to some of the object's components. This is often done using “private” and “public” access modifiers. In Python, encapsulation is achieved by prefixing an attribute or method with a double underscore (`--`), making it **private**.

Example:

```
1 class BankAccount:
2     def __init__(self, balance):
3         self.__balance = balance # Private variable
4
5     def deposit(self, amount):
6         self.__balance += amount
7
8     def get_balance(self):
9         return self.__balance
10
11 # Accessing balance through methods
12 account = BankAccount(1000)
13 account.deposit(500)
14 print(account.get_balance()) # Output: 1500
```

Explanation

This code demonstrates encapsulation by defining a class **BankAccount** that hides the internal balance attribute and provides controlled access to it through the `deposit` and `get_balance` methods. The balance cannot be directly modified; it can only be updated via the `deposit` method, ensuring secure and consistent data management.

1.2.3 Inheritance

Inheritance is a mechanism in OOP that allows a class to inherit the attributes and methods of another class. It helps in creating a new class based on an existing class, allowing code reuse and extension.

Example:

```
1 class Animal:
2     def speak(self):
3         print("Animal speaks")
4
5 class Dog(Animal):
6     def speak(self):
7         print("Woof")
8
9 dog = Dog()
```

```
10 dog.speak() # Output: Woof
```

Explanation

In this example, the Dog class inherits from the Animal class. The Dog class overrides the speak method of the Animal class to provide its own implementation, which prints "Woof". When an object of the Dog class calls speak(), the overridden method in Dog is executed, demonstrating how inheritance allows the subclass (Dog) to extend and modify the behavior of the parent class (Animal).

1.2.4 Polymorphism

Polymorphism allows methods to take on many forms. It enables a single method to behave differently based on the object it is acting upon.

Example:

```
1 class Bird:
2     def speak(self):
3         print("Tweet")
4
5 class Dog:
6     def speak(self):
7         print("Bark")
8
9 def make_sound(animal):
10     animal.speak()
11
12 bird = Bird()
13 dog = Dog()
14
15 make_sound(bird) # Output: Tweet
16 make_sound(dog) # Output: Bark
```

Explanation

This code demonstrates Polymorphism by using the make_sound function, which can operate on different object types (Bird and Dog) as long as they implement a speak method. The function dynamically calls the appropriate speak method for each object, showcasing how a common interface can handle diverse behaviors.

1.3 Sample Problem

This example demonstrates the implementation of several key object-oriented programming concepts including abstraction, encapsulation, inheritance, and polymorphism.

1.3.1 Problem Statement

Create a restaurant ordering system, where you can be able to perform below mentioned tasks:

- Create a base class **MenuItem** to represent a general menu item, encapsulating common attributes like name, price, and availability. Provide methods to retrieve and update availability while hiding the implementation details.
- Protect sensitive data, such as the availability of menu items, by making it private. Use getter and setter methods to control and validate access to this data.
- Extend the functionality of the MenuItem class by creating derived classes Food and Drink. These subclasses should include additional attributes specific to their category, such as cuisine for food and size for drinks.
- Implement a method get_details() that is defined in the base class **MenuItem** and overridden in the subclasses Food and Drink to provide customized details for each type of menu item.

1.3.2 Solution

```
1 # Abstraction and Encapsulation: Base class for Menu Item
2 class MenuItem:
3     def __init__(self, name, price, item_id, available=True):
4         self.name = name
5         self.price = price
6         self.item_id = item_id
7         self.__available = available # Private attribute for
availability
8
9     # Encapsulation
10    # Getter and Setter for availability
11    def is_available(self):
12        return self.__available
13
14    def set_availability(self, status):
15        if isinstance(status, bool):
16            self.__available = status
17        else:
18            print("Availability status must be a boolean.")
19
20    def get_details(self):
21        return f"Item: {self.name}, Price: ${self.price:.2f}, ID: {self
.item_id}"
22
23 # Inheritance: Derived class for Food
24 class Food(MenuItem):
```

```

25     def __init__(self, name, price, item_id, cuisine, available=True):
26         super().__init__(name, price, item_id, available)
27         self.cuisine = cuisine
28
29     # Polymorphism: Overriding get_details method
30     def get_details(self):
31         return (f"Food - Name: {self.name}, Price: ${self.price:.2f}, "
32               f"Cuisine: {self.cuisine}, ID: {self.item_id},
33               Available: {self.is_available()}")
34
35 # Inheritance: Derived class for Drinks
36 class Drink(MenuItem):
37     def __init__(self, name, price, item_id, size, available=True):
38         super().__init__(name, price, item_id, available)
39         self.size = size
40
41     # Polymorphism: Overriding get_details method
42     def get_details(self):
43         return (f"Drink - Name: {self.name}, Price: ${self.price:.2f}, "
44               f"Size: {self.size}, ID: {self.item_id}, Available: {self.is_available()}")
45
46 #main program
47 if __name__ == "__main__":
48     # Create objects for Food and Drink
49     pizza = Food("Pepperoni Pizza", 15.99, "F001", "Italian")
50     coffee = Drink("Latte", 4.99, "D101", "Medium")
51
52     # Display details
53     # polymorphism
54     print(pizza.get_details()) # Food-specific details
55     print(coffee.get_details()) # Drink-specific details
56
57     # Encapsulation: Check and update availability
58     print("Is the pizza available?", pizza.is_available())
59     pizza.set_availability(False) # Update availability
60     print("Is the pizza available after update?", pizza.is_available())
61
62     # inheritance
63     print("Drink Details:", coffee.get_details())

```

Explanation

- The MenuItem class serves as a base class that hides the implementation details of item availability using a private attribute `__available`. Public methods (`is_available` and `set_availability`) provide controlled access to this private attribute, demonstrating encapsulation.
- The Food and Drink classes inherit attributes and methods from the MenuItem

class, allowing them to reuse the base class functionality while adding specific attributes like `cuisine` and `size`.

- The `get_details` method is overridden in the `Food` and `Drink` subclasses to provide specific implementations for each class, allowing the same method name to exhibit different behaviors based on the object type.
- Instances of `Food` and `Drink` demonstrate the use of polymorphism through the `get_details` method and encapsulation by checking and updating item availability. Inheritance is highlighted by sharing common functionality from the `MenuItem` class.

2 Submission Problem

2.1 Instructions

- Complete the code for the following problem in a file called `main.py`.
- Test your code using the provided test cases.
- Once you're confident your code is correct, submit the `main.py` file to the Grade-scope assignment.
- Do not change name of the functions or variables.

2.2 Problem Statement - LibraryBook Class

In this exercise, you will complete the implementation of a `LibraryBook` class in Python. Below is the incomplete class definition:

```
1 class LibraryBook:
2
3     def __init__(self, title: str, author: str):
4         # constructor
5         self.title = title
6         self.author = author
7         self.is_checked_out = False
8         self.borrower = ""
9         self.renew_count = 0
10
11     def get_author(self):
12         # return the author's name
13         pass
14
15     def get_title(self):
16         # return the book's title
17         pass
18
19     def get_borrower(self):
20         # return the name of the borrower or "NONE" if the book is
21         not checked out
22         pass
23
24     def get_renew_count(self):
25         # return the number of times the book has been renewed
26         pass
27
28     def check_out(self, borrower_name: str):
```

```

28         # check out the book to the given borrower
29         pass
30
31     def renew(self):
32         # renew the book for the current borrower
33         pass
34
35     def return_book(self):
36         # return the book to the library
37         pass
38
39     def get_status(self):
40         # return True if the book is available for checkout, False
         otherwise
41         pass

```

You need to implement the following methods:

- **get.author:** This method should return the author's name of the book.
- **get.title:** This method should return the title of the book.
- **get.borrower:** This method should return the current borrower of the book. If the book is not checked out, it should return "NONE".
- **get.renew_count:** This method should return the number of times the book has been renewed by the current borrower. If the book has not been checked out, it should return 0.
- **check_out:** This method is called when someone wants to check out the book. If the book is already checked out, it should return -1; otherwise, it should update the book's status and return 1.
- **renew:** This method is called when the current borrower wants to renew the book. If the book has not been checked out, return -1. If successful, increment the renew count and return 1.
- **return_book:** This method is called when the borrower wants to return the book. If the book is not checked out, return -1; otherwise, return the book and return 1.
- **get.status:** This method should return **True** if the book is available for checkout, otherwise return **False**.

2.2.1 Test Case

Input

```
1 Invisible—Man Ralph—Ellison      # [title] [author] for the
   constructor
2 author?                          # calls the get_author function
3 title?                           # calls the get_title function
4 borrower?                        # calls the get_borrower function
5 check_out aditya                 # calls the check_out function,
   second word is borrower
6 borrower?                        # calls get_borrower function
7 status?                         # calls get_status function
8 renew                           # calls the renew function
9 renew_count?                    # calls get_renew_count function
10 return                          # calls return_book function
11 status?                        # calls get_status function
12 END                             # marks the end of the input and
   terminates the program
```

Output

```
1 Ralph—Ellison
2 Invisible—Man
3 NONE
4 1                                # checkout successful
5 aditya
6 NOT AVAILABLE
7 1                                # renew successful
8 1                                # renewed once after being issued
9 1                                # return successful
10 AVAILABLE
```

Additional Questions

- How do you create an object of a class in Python?
- What are public and private variables or methods in Python, and how do they differ?
- What are the different types of constructors in Python?
- Can you think of other relevant methods or variables for this problem?

3 Practice Problems

3.1 Complex Calculator

3.1.1 Problem Statement

Design a class `Calculator` that performs basic arithmetic operations and supports additional operations like exponentiation and square root. The class should encapsulate the following private attributes:

- `_last_result`: Stores the last result computed.

The class should include the following methods:

- `add(a, b)`: Returns the sum of `a` and `b`.
- `subtract(a, b)`: Returns the result of `a` minus `b`.
- `multiply(a, b)`: Returns the result of `a` multiplied by `b`.
- `divide(a, b)`: Returns the result of `a` divided by `b` (throws an exception if `b == 0`).
- `exponentiate(a, b)`: Returns `a` raised to the power of `b`.
- `squareroot(a)`: Returns the square root of `a`.
- `get_last_result()`: Returns the last computed result.
- `clear()`: Resets the `_last_result` to 0.

Additionally, implement polymorphism to handle operations on both integers and floating-point numbers.

3.1.2 Test Case 1

Main Program

```
1 calc = Calculator()
2 calc.add(10, 5)
3 calc.subtract(10, 3)
4 calc.multiply(2, 3)
5 calc.divide(10, 0)
6 calc.exponentiate(2, 3)
7 print(calc.get_last_result())
```

Output

```
1 Result: 8
```

Explanation

Operations like addition, subtraction, multiplication, and exponentiation update the result stored in `_last_result`, which can be accessed via the `get_last_result()` method. Handling division by zero should raise an exception.

3.2 Event Calendar

3.2.1 Problem Statement

Create a class `EventCalendar` that manages events scheduled on different dates. The class should have the following private attributes:

- `_events`: A dictionary where keys are dates (in the format `YYYY-MM-DD`) and values are lists of event descriptions.

The class should include the following methods:

- `add_event(date, event)`: Adds an event to the specified date.
- `remove_event(date, event)`: Removes a specific event from a specified date.
- `get_events(date)`: Returns a list of all events scheduled on a given date.
- `get_all_events()`: Returns a dictionary of all events scheduled across all dates.
- `clear()`: Clears all events.
- `__str__()`: Returns a string representation of the calendar.

Ensure that no duplicate events are added for the same date.

3.2.2 Test Case 1

Main Program

```
1 calendar = EventCalendar()
2 calendar.add_event("2024-12-01", "Meeting with John")
3 calendar.add_event("2024-12-01", "Conference Call")
4 calendar.add_event("2024-12-02", "Team Dinner")
5 calendar.remove_event("2024-12-01", "Conference Call")
6 print(calendar.get_events("2024-12-01"))
```

Output

```
1 [ 'Meeting with John ' ]
```

Explanation

The event “Conference Call” was removed from the list of events scheduled for “2024-12-01”. The remaining event is “Meeting with John”.

3.3 Chess Game

3.3.1 Problem Statement

Create a class-based implementation for a simple chess game.

- You need to represent the chessboard, pieces, and moves.
- The chessboard should be represented as an 8x8 grid, and each piece should be represented as a subclass of a base class `Piece`.
- The base class should have common methods for movement and capturing, while each piece subclass (e.g., `King`, `Queen`, `Knight`) should define its unique movement logic.

The following methods should be implemented:

- `move(from_position, to_position)`: Moves a piece from one position to another if the move is valid.
- `capture(piece)`: Captures another piece.
- `get_valid_moves(position)`: Returns a list of all valid moves for a piece from the given position.
- `is_checkmate()` or `is_stalemate()`: Determines if the current player is in checkmate or stalemate.

3.3.2 Test Case 1

Main Program

```
1 board = ChessBoard()
2 king = King()
3 queen = Queen()
4 board.place_piece(king, (0, 4)) # Place king at position (0,4)
5 board.place_piece(queen, (4, 4)) # Place queen at position (4,4)
6 king.move((0, 4), (1, 4)) # King moves
7 queen.capture(king)
8 print(board.is_checkmate())
```

Output

```
1 True
```

Explanation

The King was captured by the Queen, resulting in a checkmate condition, which is detected by the `is_checkmate()` method.

3.4 Car Rental Management System

3.4.1 Problem Statement

Design a class `CarRental` that manages a fleet of cars available for rent. Each car is represented by a `Car` class, and customers are represented by a `Customer` class. The system should support the following features:

- **Attributes:**

- **CarRental Class:**
 - * `_cars`: A list of cars in the fleet.
 - * `_rented_cars`: A dictionary mapping customers to their rented cars.
- **Car Class:**
 - * `car_id`: A unique identifier for the car.
 - * `brand`: Brand of the car.
 - * `model`: Model of the car.
 - * `rental_price`: Price per day for renting the car.
 - * `available`: A boolean indicating if the car is available for rent.
- **Customer Class:**
 - * `name`: Name of the customer.
 - * `customer_id`: A unique identifier for the customer.
 - * `rented_cars`: A list of cars rented by the customer.

- **Methods:**

- **CarRental Class:**
 - * `add_car(car)`: Adds a new car to the fleet.
 - * `remove_car(car_id)`: Removes a car from the fleet.
 - * `rent_car(customer, car_id)`: Allows a customer to rent a car.
 - * `return_car(customer, car_id)`: Allows a customer to return a rented car.
 - * `list_available_cars()`: Lists all available cars for rent.
 - * `list_rented_cars()`: Lists all currently rented cars and their customers.
- **Customer Class:**
 - * `rent_car(car)`: Adds a car to the customer's rented cars.
 - * `return_car(car)`: Removes a car from the customer's rented cars.
- **Inheritance:** Create subclasses of `Car` like `SUV`, `Sedan`, and `Truck`, each with unique attributes (e.g., `cargo_capacity` for `Truck`, `luxury_rating` for `Sedan`).

3.4.2 Test Case 1

Input:

```
1 # Create CarRental system
2 rental = CarRental()
3
4 # Add cars to the fleet
5 car1 = SUV("SUV001", "Toyota", "RAV4", 70, 500)
6 car2 = Sedan("SED001", "Mercedes", "C-Class", 120, 5)
7 car3 = Truck("TRK001", "Ford", "F-150", 90, 1000)
8
9 rental.add_car(car1)
10 rental.add_car(car2)
11 rental.add_car(car3)
12
13 # Create a customer
14 customer = Customer("John Doe", "C001")
15
16 # Rent a car
17 rental.rent_car(customer, "SUV001")
18
19 # List rented cars
20 print(rental.list_rented_cars())
21
22 # Return the car
23 rental.return_car(customer, "SUV001")
24
25 # List available cars
26 print(rental.list_available_cars())
```

Output:

```
1 Rented Cars:
2 John Doe: [Toyota RAV4]
3
4 Available Cars:
5 ['Toyota RAV4', 'Mercedes C-Class', 'Ford F-150']
```